Direct Evolution Monitoring Portal

TEAM TAHOE

2025 -2026

School of Biological Sciences

Queen Mary University of London

BIO727P

MSc Bioinformatics – Software Development Group Project

Direct Evolution Monitoring Web Portal

Technical Documentation Report

GitHub Link: https://github.com/ccharles22/Tahoe-Project-2026

GROUP MEMBERS:

Candice Charles

Luke Oakarr

Maria Paola Luciani

Reman Puvaneswaran

Patricia Arakit-Osire

Queen Mary University of London

2025/26

Table of Contents

1. Introduction *4*

Project Overview **4**

Scientific Background **4**

2. Software Architecture and Design Philosophy *4*

2.1 Overall Design Philosophy **4**

Scientific Correctness as the Primary Constraint 5

Modularity and Separation of Concerns 5

User-Centred Simplicity 5

Robust Security 5

2.2 Database Schema **5**

2.2.1 Overview 5

Core Biological and User Tables 6

Mutations and Lineage Infrastructure 6

Quantitative Metric Layer 6

Sequence Processing Layer 7

Analytical and Visualisation Infrastructure 7

3. Technologies Used *7*

3.1 Web Framework - Flask **7**

Selection Rationale 7

How It Was Used 7

3.2 Database - PostgreSQL **8**

Selection Rationale 8

How It Was Used 8

PostgreSQL Deployment and secure remote access via Tailscale 8

3.3 Frontend Technologies **9**

3.4 Bioinformatics Libraries **9**

4. Core Functionality *10*

4.1 User Registration and Authentication **10**

Registration Workflow 10

Login and Session Management 10

Password Security 10

4.2 Staging: Protein Selection and Plasmid Validation **10**

Overview 10

Stage 2 -Step 1: Fetch WT from UniProt 11

Stage 2 -Step 2: Upload plasmid FASTA and validate 11

4.3 Experimental Data Parsing and Quality Control **12**

File Parsing 12

Dynamic Schema Handling 13
Quality Control Rules 13

4.4 Analysis and Report Generation 15
4.4.1 Sequence Processing: DNA to Protein Translation 15
Variants that trigger QC flags are not discarded; instead, the experiment status is marked as ANALYSED_WITH_ERRORS, ensuring that potentially meaningful biological observations remain available for researcher review. 16
4.4.2 Mutation Identification and Classification 16
- Codon-by-Codon Comparison 16
- Indel Detection via Protein Alignment 17
- Mutation Aggregation 17

4.4.3 Unified Activity Score 17

4.5 Data Segregation and User-Specific Access Control 18

5. Visualisations 19

5.1 Top Performers Table 19

5.2 Activity Score Distribution by Generation Plot 19

5.3 Variant Lineage Visualisation 20

5.4 Protein Similarity Network (Mutation Co-occurrence) 20

5.5 Bonus Visualisations: Multi-Scale Analytical Narrative 22
5.5.1 Phenotype - Space level: 3D activity landscape 22
5.5.2 Domain level: Domain enrichment heatmap 22
5.5.3 Variant Level : Mutation fingerprinting 23
5.5.4 Residual Level: Mutation Frequency 23

6. Limitations of the Software 24

6.1 Technical Limitations and Short-Term Improvements 24

6.3 Scalability Considerations and Infrastructure Roadmap 26
Infrastructure 26
Computational Throughput 26
Visualisation Performance 25
Advanced Analytics 25

7. Conclusion 26

8. References 27

1. Introduction

Project Overview

The Direct Evolution Monitoring Portal is a full-stack bioinformatics web application developed to support researchers in managing and interpreting directed protein evolution experiments. Built by Team Tahoe as part of the BIO727P Software Development Group Project at Queen Mary University of London.

The portal guides users through a complete directed evolution workflow: selecting a wild-type (WT) protein via UniProt accession retrieval, uploading and validating the corresponding plasmid construct with circular DNA support, ingesting multi-generation experimental data in TSV or JSON format, and analysing variant sequences to produce ranked performance outputs and visualisations.

All data is underpinned by a curated PostgreSQL relational database organised around the hierarchy: Users, Wild-Type Proteins, Experiments, Generations, and Variants. Evolutionary lineage is captured through parent-child variant relationships and a closure table for efficient ancestry queries, while a flexible metrics system stores raw, normalised, and derived values alongside standardised metric definitions.

Scientific Background

Directed evolution mimics natural selection in a laboratory setting to engineer proteins with improved or entirely new properties. The process involves iterative rounds of mutagenesis, expression, and selection, progressively accumulating beneficial mutations over successive generations. Each round produces a library of sequence variants differing from the parental wild-type by one or more nucleotide changes, insertions, or deletions.

Key informatics requirements include accurate tracking of DNA and protein sequences across generations; precise classification of mutations relative to a wild-type reference; quantification of functional performance through measurable outputs such as DNA and protein yield; and correlation of sequence-level changes with functional outcomes. The portal integrates established bioinformatics resources including the UniProt protein database, the BLOSUM62 substitution matrix, and standard genetic code translation tables, ensuring analytical results are grounded in accepted molecular biology principles.

2. Software Architecture and Design Philosophy

2.1 Overall Design Philosophy

The portal was designed around four guiding principles that shape every technical decision in the system.

Scientific Correctness as the Primary Constraint

Because the platform processes biological sequence data that directly informs research conclusions, accuracy takes precedence over convenience at every stage. Plasmid validation is conservative by default, requiring exact sequence matches before confirming a construct. Mutation classification uses biochemically grounded BLOSUM62 scoring with affine gap penalties. Activity score normalisation explicitly records the strategy applied so results remain interpretable and auditable. Quality control flags are raised rather than silently suppressed, and partial analysis results are preserved alongside error indicators rather than discarded.

Modularity and Separation of Concerns

The application is structured as loosely coupled components, each responsible for a distinct stage of the workflow. Flask blueprints organise the web layer into logical areas - authentication, experiment staging, data ingestion, analysis, and visualisation - while dedicated service modules encapsulate the bioinformatics logic independently of the routing layer. This separation allows individual components such as the plasmid validator, mutation caller, or activity score calculator to be tested and modified without changes elsewhere in the system.

User-Centred Simplicity

The interface guides researchers through a sequential workflow, protein selection, plasmid upload, data ingestion, analysis, and visualisation, without requiring familiarity with the underlying algorithms. Error messages are actionable and row-indexed, helping users locate and correct problems efficiently. Where technical decisions have been made on behalf of the user, those choices are documented and parameters stored in the database for reproducibility.

Robust Security

Authentication and data access follow a defence-in-depth model. Passwords are hashed with bcrypt using adaptive work factors, sessions are protected with HttpOnly and SameSite cookies, and all database retrieval operations are filtered by authenticated user identity to ensure data isolation between users.

2.2 Database Schema

2.2.1 Overview

The database schema was designed to support the full lifecycle of directed evolution experiments, from sequence definition and experimental setup to lineage tracking, quantitative analysis, and visualisation. The schema is organized around the workflow hierarchy:

Users → Wild-Type Proteins → Experiments → Generations → Variants ,

Each user manages wild-type protein entries (stored with UniProt identifiers, protein sequences, and associated plasmid constructs), while linking experiments containing multiple generations, reflecting iterative rounds of mutagenesis and selection, thereby allowing

traceable ownership of all records. All tables use UUID primary keys for entity integrity and employ foreign keys to maintain relational consistency across proteins, variants, experiments, and metrics.

Core Biological and User Tables

Table name	Purpose	Key relationships
users	Stores user credentials and account metadata.	Referenced by experiments, wild_type_proteins.
wild_type_proteins	Stores UniProt-derived WT protein sequences and associated metadata.	FK → users; referenced by experiments, protein_features, wild_type_controls.
protein_features	Stores UniProt domain and site annotations.	FK → wild_type_proteins.
experiments	Represents a directed evolution campaign.	FK → users, wild_type_proteins; referenced by generations, experiment_metadata, embedding_runs.
experiment_metadata	Stores dynamic ingestion metadata (TSV/JSON fields not in core schema).	FK → experiments.
generations	Represents iterative rounds of evolution within an experiment.	FK → experiments; referenced by variants, wild_type_controls, metrics.
variants	Stores engineered plasmid variants within a generation.	FK → generations; self-FK → variants (parent-child lineage). Referenced by multiple analysis tables.

Mutations and Lineage Infrastructure

Table name	Purpose	Key relationships
mutations	Stores DNA and protein mutation records per variant.	FK → variants; referenced by downstream analysis tables.
mutation_introduction_events	Tracks when and how mutations were introduced during directed evolution.	FK → variants, possibly generations.
variant_lineage_closure	Precomputed ancestor–descendant relationships between variants.	FK → variants (ancestor & descendant).

Quantitative Metric Layer

Table name	Purpose	Key relationships
wild_type_controls	Stores WT baseline measurements per generation.	FK → generations, wild_type_proteins; referenced by metrics.

metrics	Unified storage for raw, normalised, and derived measurements.	FK → variants, generations, wild_type_controls, metric_definitions.
metric_definitions	Lookup dictionary defining metric names and units.	Referenced by metrics.

Sequence Processing Layer

Table name	Purpose	Key relationships
variant_sequence_analysis	Stores ORF mapping, strand, QC flags, translation, and mapping metadata.	FK → variants; referenced by mutations.

Analytical and Visualisation Infrastructure

Table name	Purpose	Key relationships
embedding_runs	Stores metadata for dimensionality reduction runs (e.g., PCA, t-SNE).	FK → experiments; referenced by embedding_points.
embedding_points	Stores low-dimensional coordinates for each variant in a specific embedding run.	FK → embedding_runs, variants.
variant_performance_rankings	Stores precomputed ranking of variants based on selected metrics.	FK → variants, experiments, metrics.

3. Technologies Used

3.1 Web Framework - Flask

Selection Rationale

Flask was selected as the web framework due to its lightweight and modular nature, which aligns well with the project's separation-of-concerns philosophy. Unlike heavier frameworks, Flask does not impose a rigid project structure, allowing the team to organise the codebase into purpose-built blueprints. Flask's extensive ecosystem - including Flask-Login, Flask-WTForms, and Flask-Bcrypt - provided production-ready implementations of security-critical components without requiring custom development.

How It Was Used

The application is organised into Flask blueprints corresponding to distinct workflow areas. All HTML pages are generated server-side templating. A base layout template defines the shared navigation, header, and flash message container, extended by page-specific templates for each workflow step. Conditional template logic shows or hides interface elements based

on experiment state, for example displaying plasmid validation results only after a successful upload.

3.2 Database - PostgreSQL

Selection Rationale

PostgreSQL was chosen as the primary relational database system for its advanced analytical capabilities and strong support for complex hierarchical data. Considering the complex hierarchical biological relationships of experiments, generations, variants and lineage, as well as thorough quantitative measurements, the relational database provides great transactional consistency and key constraints for scientific data integrity.

The citext extension enables case-insensitive unique identifiers for user credentials. Native JSONB fields allow flexible storage of heterogeneous experimental metadata without frequent schema changes. PL/pgSQL triggers enable automatic maintenance of derived fields including timestamps and generation relationships, reducing application-side logic.

How It Was Used

The database served as the single source of truth for all biological, experimental, and analytical data. Mutations are recorded at both DNA and protein levels for each variant. Quantitative measurements: including raw, normalized, and derived values such as activity scores, are stored in a unified metrics table.

To support efficient lineage analysis, a closure table is maintained that stores ancestor–descendant relationships between variants. This enables fast retrieval of complete evolutionary paths without recursive queries. Additional tables store wild-type controls, experiment metadata, and sequence analysis results generated by downstream computational pipelines.

For visualization and performance optimization, materialized views are used to provide denormalized datasets for common analytical tasks such as activity landscape plotting and domain-level mutation enrichment. These views are refreshed after data loading or analysis

PostgreSQL Deployment and secure remote access via Tailscale

The PostgreSQL database was installed locally on a host machine and shared securely across the team via Tailscale, a zero-configuration VPN that assigns private IP addresses without exposing the database to the public internet. Each team member connected using a shared project role via a DATABASE_URL environment variable. An optional logical replication subscriber was configured locally to support heavy queries and offline analysis without impacting the shared server.

To allow remote access without exposing the database to the public internet, Tailscale, a zero-configuration VPN creating an encrypted private overlay network, was used to assign a private IP address to each device by:

- Updating postgresql.conf to allow connections (listen_addresses = '*')
- Adding authentication rules in pg_hba.conf to permit connections from the Tailscale address range (100.64.0.0/10)
- Restarting the PostgreSQL service to apply changes

Each team member was able to connect to the database using the host machine's Tailscale IP address with directed authorization:

postgresql://<username>:<password>@<TAILSCALE_IP>:5432/bio727p_group_project

3.3 Frontend Technologies

Technology	Purpose	Rationale
Jinja2 Templates	Server-side HTML generation	Seamless Flask integration; consistent rendering across sessions
HTML5 / CSS3	Page structure and styling	Functional, uncluttered design suited to scientific context
Flask-WTForms	Form handling and CSRF protection	Built-in validators, automatic CSRF tokens, field-level error messages
Plotly	Interactive supplementary visualisations	Enables zooming, panning, hover tooltips and 3D rotation client-side
Matplotlib / NetworkX	Static publication-ready figures	Reproducible server-side rendering stored in analysis_MPL/plots
Vanilla JavaScript	Lightweight client-side interactions	Minimal dependency footprint; no framework complexity or build toolchains

3.4 Bioinformatics Libraries

Library	Purpose
Biopython (PairwiseAligner)	Six-frame protein translation and local/global alignment with BLOSUM62
BLOSUM62 matrix	Biochemically informed scoring of amino acid substitutions in mutation calling
Pandas / NumPy	Data preparation, aggregation and structuring prior to visualisation
scikit-learn	PCA and t-SNE dimensionality reduction for protein embedding visualisations

4. Core Functionality

4.1 User Registration and Authentication

The authentication system follows a security-first design, implementing defence in depth across registration, login, session management, and password storage.

Registration Workflow

Registration proceeds through three stages. First, WTForms validates input client- and server-side, checking email format, password confirmation match, minimum length of 8 characters, and field completeness. Second, the system queries the database to ensure both username and email are unique, catching conflicts at both the application and database levels via IntegrityError handling. Third, a new User record is created with a UUID primary key and a bcrypt-hashed password, after which Flask-Login immediately authenticates the new user for a seamless experience.

UUIDs were chosen over auto-incrementing integers to prevent user enumeration attacks, to support potential future distributed deployments, and to avoid leaking registration order through public-facing identifiers.

Login and Session Management

Login uses multi-stage verification. The system retrieves the user record by username, then verifies the submitted password against the stored bcrypt hash using constant-time comparison. Generic error messages are used for both non-existent users and incorrect passwords to prevent username enumeration. A known timing vulnerability exists: when a username does not exist, bcrypt is skipped, producing a faster response. Implementing a dummy bcrypt check on failed lookups is documented as a short-term improvement.

Password Security

Bcrypt was selected over PBKDF2 because it is specifically designed for password hashing, incorporating built-in random salting, an adaptive work factor (default: 12, yielding approximately 100-300ms per hash on modern hardware), and resistance to parallelised GPU attacks. The stored hash encodes the algorithm version, work factor, salt, and computed hash in a self-describing format, enabling future algorithm migration without maintaining separate metadata tables.

4.2 Staging: Protein Selection and Plasmid Validation

Overview

Stage 2 initiates an experiment using a user-selected wild-type (WT) protein and establishes the baseline required for downstream analyses. It consists of two steps: (1) retrieving the WT amino-acid sequence and associated feature annotations from UniProt, and (2) uploading and validating the corresponding plasmid FASTA to confirm that the plasmid encodes the selected WT protein. A critical requirement of plasmid validation is support for plasmid circularity, including cases where the coding region spans the end of the linear FASTA representation and continues at the beginning across the plasmid origin. Stage 2, therefore, acts as a quality gate: if the WT baseline is incorrect, downstream mapping and mutation calling are not scientifically reliable.

Stage 2 -Step 1: Fetch WT from UniProt

Step 1 starts when the user submits a UniProt accession to stage a WT protein for an experiment. The system retrieves the WT amino-acid sequence and extracts feature annotations (e.g., domains) from UniProt for later interpretation and visualisation. UniProt retrieval uses a fixed timeout and retry logic to handle transient failures such as rate limiting or server errors, and failures are surfaced to the user explicitly rather than failing silently.

This step is designed to allow the experiment setup to proceed even when the WT plasmid sequence is not yet available. Immediately after a successful UniProt fetch, the system generates a placeholder coding sequence by back-translating the WT protein. Codons are chosen randomly to preserve a valid-length CDS without implying biological ground truth, and unknown or ambiguous residues are represented as NNN codons to preserve frame length without unsupported assumptions. The placeholder is stored as experiment-scoped metadata and is intended to be overwritten when the real plasmid FASTA is uploaded in Step 2.

WT protein records have persisted using an upsert strategy keyed by UniProt accession, avoiding duplication across experiments while maintaining a stable canonical reference sequence per accession. Plasmid sequence is intentionally treated as experiment-scoped rather than WT-scoped state, since plasmid constructs can differ between experiments even when the same WT is used. Once staging completes, the WT is linked to the user's experiment context; staging can attach to an existing experiment or create a new one, with conservative naming behaviour to avoid overwriting established experiment identities. Dependent outputs are then marked as stale, and any cached WT mapping state is cleared where applicable, preventing downstream stages from reusing results computed against an outdated WT baseline. UniProt feature annotations are inserted when missing and are not automatically refreshed once present, which is documented as a limitation and a candidate for controlled refresh in future work.

Stage 2 -Step 2: Upload plasmid FASTA and validate

Step 2 begins when an authenticated user uploads a plasmid FASTA file to associate a WT plasmid sequence with an experiment. Access control is enforced so that only the experiment owner can modify the plasmid state.

Uploaded FASTA files are parsed using a strict single-record parser. Input is normalised, including converting U to T, and bases are validated against the extended IUPAC alphabet (ACGTNRYWSKMBDHV). Ambiguity codes are accepted because plasmid exports often include uncertainty characters (e.g., N), while non-IUPAC characters are rejected early to avoid data corruption and ambiguous downstream behaviour. If parsing fails, the upload is rejected, and a user-facing error is shown.

If parsing succeeds, the plasmid is validated before becoming the experiment's WT plasmid. Validation is circularity-aware and orientation-agnostic. Circularity is handled by searching across an origin-spanning sequence, equivalent to analysing DNA + DNA, which allows detection of coding regions that cross the plasmid origin. Orientation is handled by analysing both the forward strand and the reverse complement. The system translates all six reading

frames across both strands, reducing false negatives from unknown frame offsets and making the method robust across cloning configurations.

Translation uses a shared default genetic code (Table 11), and the code table used is reported in each validation result (genetic_code_used). Standardising translation semantics prevents inconsistencies between plasmid validation and later stages; however, the bacterial default limits applicability outside bacterial systems and motivates future support for selectable genetic code tables.

Validation occurs in two phases. First, the system searches for an exact match of the full WT protein sequence across all translated frames, providing a fast and unambiguous acceptance path. If no exact match is found, the system runs a local alignment to identify the best approximate hit and compute's identity and coverage for diagnostics. By default, approximate matches do not pass validation even when identity and coverage are high, because accepting an incorrect WT baseline would cascade errors into downstream mapping and mutation calling. A tolerant mode using configurable identity/coverage thresholds is a suitable future enhancement, but strict mode remains the default to preserve scientific integrity at staging.

For each validation run, the system records validation status, best-hit identity and coverage, strand and reading frame, nucleotide start/end coordinates, and whether the coding region wraps across the plasmid origin. Results are saved for auditability and reproducible debugging: a compact summary is cached for responsive UI rendering, and a database record is written for each attempt. Coordinates are reported using a 0-based start index and a 0-based inclusive end index, and wraps=True indicates origin-spanning matches; this convention is documented explicitly because many bioinformatics tools use half-open intervals.

4.3 Experimental Data Parsing and Quality Control

File Parsing

The service accepts TSV/CSV and JSON files (allowed extensions: .tsv, .csv, .json). The parser requires a header row and automatically detects the delimiter from the first line. Files larger than 50 MB are rejected. Parsing errors occur for empty files, header-only files, or non-UTF-8 encoding. The JSON parser accepts an array of objects, an object with a records or variants key, or a single object. JSON syntax errors are reported with line and column numbers to aid debugging.

Extracted values are normalised: DNA sequences are converted to uppercase with whitespace removed; numeric columns are coerced where possible; completely empty rows are skipped. Parsers expose records, detected_fields, errors, warnings, and get_summary() which returns total_records, detected_fields, error_count, and warning_count.

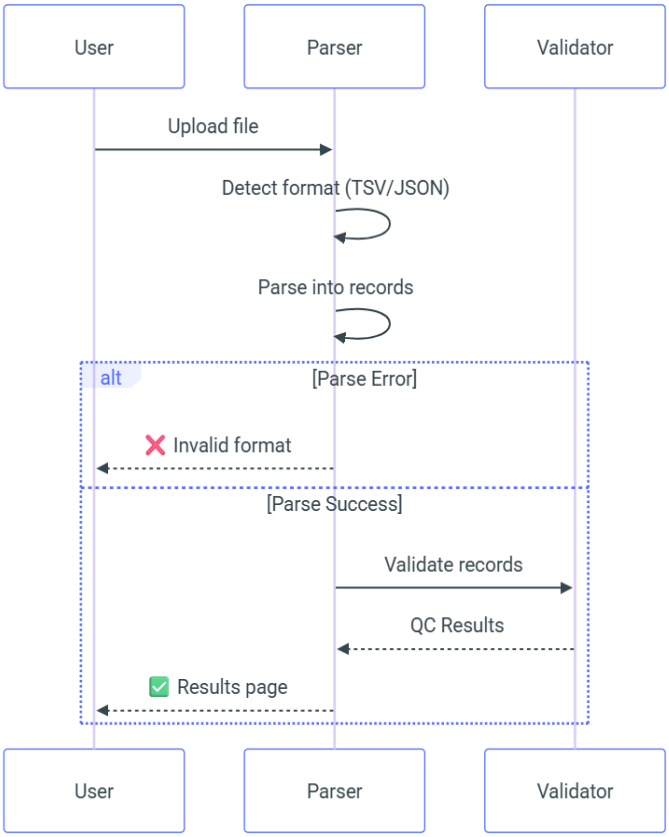

Figure 1. Sequence diagram showing a user uploading a file to a parser that detects the format (TSV/JSON) and parses records, followed by validation and QC results if parsing succeeds, or an invalid format error if parsing fails.

Dynamic Schema Handling

The FIELD_MAPPING configuration in config.py handles field name heterogeneity. BaseParser.normalise_field_names() maps different source headers to standard internal field names -for instance, plasmid_variant_index maps to variant_index, and dna_quantification_fg maps to dna_yield. This layer separates external data conventions from the internal schema, ensuring downstream validation and persistence logic operates consistently regardless of source terminology. Extra columns not recognised as core fields are preserved as metadata for auditing or future use, allowing strict validation and flexible data capture to occur simultaneously.

The required fields are variant_index, generation, assembled_dna_sequence, dna_yield, and protein_yield. The upload endpoint checks for their presence on a per-record basis and sends record-level validation errors for missing required fields (see routes.py).

Quality Control Rules

Check	Type	Rule
Required fields	Error (blocks upload)	variant_index, generation, assembled_dna_sequence, dna_yield, protein_yield must be present

Type validation	Error	variant_index and generation must be integers; yields must be numeric
Yield warning thresholds	Warning	Values outside 50–2000 flagged; percentile mode uses p1–p99 thresholds (min 30 samples)
DNA yield critical limits	Error	Values outside 300–5000 immediately rejected
Protein yield critical limits	Error	Values outside 20–2000 immediately rejected
Sequence length	Error	Min 100 bp, max 10000 bp; invalid characters produce errors
Frameshift check	Warning (opt-in)	Length not divisible by 3; disabled by default

All messages are row-indexed (e.g., Row 12: CRITICAL - DNA yield (45.2) below critical minimum (300.0)) to enable targeted correction. Records with errors are blocked from upload; records with warnings upload but are flagged in the response summary.

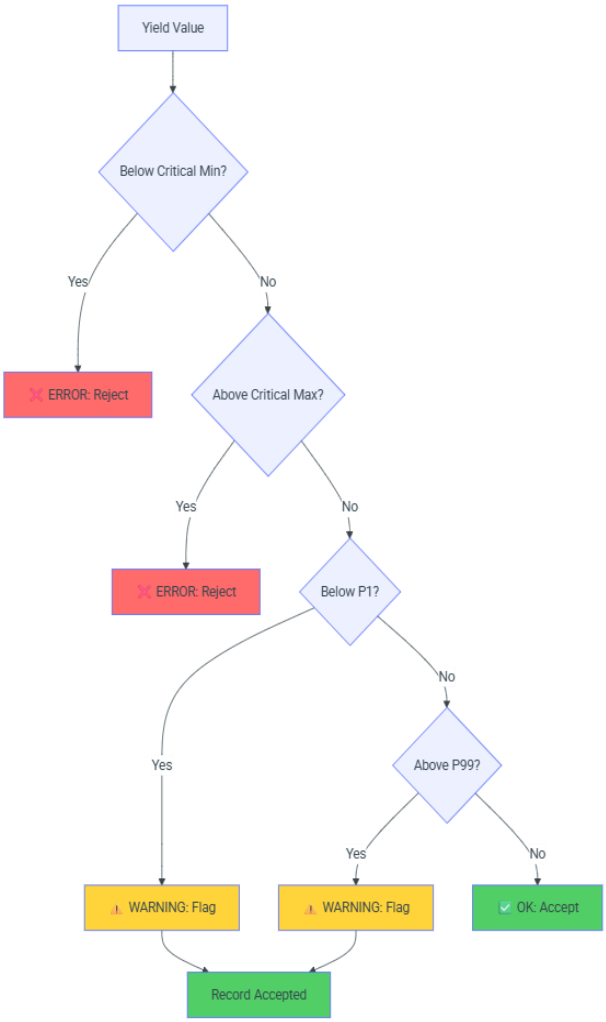

Figure 2 . Flowchart illustrating yield value validation where values outside critical minimum or maximum are rejected, values outside percentile thresholds (P1/P99) are flagged with warnings, and values within acceptable limits are accepted.

4.4 Analysis and Report Generation

4.4.1 Sequence Processing: DNA to Protein Translation

The sequence processing pipeline converts plasmid DNA sequences into protein sequences and identifies the coding region corresponding to the wild-type (WT) protein. Since plasmids are circular, the gene of interest may span the start and end boundary of a linear FASTA representation. Furthermore, the coding sequence can occur on either strand and in any of three reading frames. To ensure accurate gene identification, the pipeline evaluates all six possible reading frames—three forward and three reverse.

Sequence comparison is performed using Biopython's PairwiseAligner with the BLOSUM62 substitution matrix and affine gap penalties (gap open −10, gap extend −0.5). Substitution matrices score amino acid replacements based on evolutionary similarity, enabling biologically meaningful protein comparisons. Affine gap penalties treat insertion or deletion events as single evolutionary occurrences.

Wild-type gene mapping is carried out by the function map_wt_gene_in_plasmid(). To account for the circular nature of plasmids, the DNA sequence is concatenated with itself, preserving genes that span the origin during analysis. Each of the six translated reading frames is aligned to the WT protein sequence using Smith–Waterman local alignment, which identifies the highest-scoring matching subsequence without penalising unrelated vector backbone translations.

The best-scoring alignment determines the strand orientation, reading frame, coding sequence (CDS) coordinates, translated WT protein sequence, and sequence identity score. These coordinates are stored and reused for all experimental variants, ensuring consistent CDS extraction and avoiding repeated mapping computations.

For each plasmid variant, the CDS is extracted using the same strand, frame, and coordinate boundaries established during WT mapping. A circular slicing utility manages genes that wrap across the plasmid origin. The CDS is then translated codon-by-codon and assessed using three quality-control checks stored in a QCFlags structure.

QC Flag	Conditions Detected	Biological Significance
has_ambigous_bases	Coding sequence contains non-ACGT characters(e.g. N,R,Y)	Indicates sequencing errors or low-confidence base calls; affected codons cannot be reliably translated
has_frameshift	Coding sequence length is not divisible by three	An insertion or deletion has shifted the reading frame, producing an incorrect protein from the point of indel onwards
has_premature_stop	A stop codon occurs before the expected end of the protein	The protein is likely truncated and non-functional; may indicate a

		nonsense mutation or sequencing artefact

Variants that trigger QC flags are not discarded; instead, the experiment status is marked as ANALYSED_WITH_ERRORS, ensuring that potentially meaningful biological observations remain available for researcher review.

4.4.2 Mutation Identification and Classification

Mutations relative to the WT sequence are identified using the function call_mutation_against_wt(), which selects one of three analysis strategies depending on the properties of the coding sequence.

Condition	Strategy	Rationale
Either CDS not divisible by 3	Single FRAMESHIFT record	Reading frame disrupted; codon comparison impractical
WT and variant CDS same length	Codon-by-codon comparison	Fast and exact; each codon maps directly to its WT counterpart
CDS lengths differ (both in-frame)	Needleman-Wunsch global alignment + BLOSUM62	Indels alter length; gap-aware alignment needed for correct indel positions

Pairwise alignment was chosen over multiple sequence alignment (MSA) because the analysis is based on variant-to-reference comparisons. Pairwise comparison is computationally efficient and adequate for mutation detection, whereas MSA primarily benefits downstream analyses, such as identifying convergent mutations across the entire variant library.

Codon-by-Codon Comparison

When the WT and variant CDS lengths are identical, codons are compared sequentially. Differences are translated and categorised as follows:

- Synonymous – codon change does not alter the encoded amino acid
- Nonsynonymous – codon change alters the amino acid sequence
- Nonsense – variant codon introduces a premature stop codon
- Ambiguous – codon contains non-ACGT bases and cannot be translated reliably

Synonymous mutations do not alter the protein sequence, whereas nonsynonymous substitutions result in amino acid changes that may impact enzyme function. Nonsense mutations cause truncated proteins due to premature termination.

- Indel Detection via Protein Alignment

If the WT and variant CDS lengths differ but are both divisible by three, the sequences are translated into proteins and aligned using Needleman–Wunsch global alignment with the BLOSUM62 matrix and affine gap penalties. As both sequences represent the same gene boundaries, global alignment ensures a comprehensive comparison of the entire protein sequences. Gap characters denote insertion or deletion events as follows:

- Gap in WT alignment → insertion in variant
- Gap in variant alignment → deletion in variant

Each mutation is documented with its type, codon position, WT and variant codons, corresponding amino acids, and relevant annotation notes.

- Mutation Aggregation

For each variant, individual mutation records are consolidated into a MutationCounts structure, which includes the total numbers of synonymous mutations, nonsynonymous mutations (including nonsense), and the overall mutation count. These aggregated values are stored alongside detailed mutation records, facilitating downstream analyses such as calculating mutation frequency by position, domain, or generation without the need to reprocess the original sequences.

4.4.3 Unified Activity Score

Variant performance is consistently computed across experiments through the activity score, a single derived metric that relates DNA output to protein expression. The calculation is performed in 3 steps:

1. Normalization of raw measurements, from **DNA and protein yields (raw)** first obtained from experimental measurements across all generations

2. Computation of the final ratio using one of two strategies (depending on data availability):
   - **Wild-type normalization (preferred):** if valid WT- control measurements exist for every generation in the experiment, each variant's values are normalised relative to the WT of the same generation
     - dna_yield_normalised = dna_yield_raw / dna_yield_WT
     - protein_yield_normalised = protein_yield_raw / protein_yield_WT
   - **Generation-median normalization (fallback):** if WT controls are missing or incomplete, the median value of all variants within the same generation is used for normalization to replace the dna_yield_WT and protein_yield_WT.

3. The final activity score is computed as: activity_score = dna_yield_normalised / protein_yield_normalised.

Taking the WT control baseline as 1.0: scores above 1 indicate improved functional efficiency; scores near 1 indicate performance similar to baseline; scores below 1 indicate reduced efficiency. The activity score is used as the main performance indicator for experiment-level summaries and visual outputs, including the Top-10 variants table.

4.5 Data Segregation and User-Specific Access Control

Multi-user data segregation ensures isolated, secured, and user-only accessible data across the platform. Each wild-type protein addition is directly associated with a specific user via the user_id foreign key. All entities -experiments, generations, variants, mutations, metrics, and analysis results -are directly or indirectly linked to the owning user through this relationship chain, ensuring users can only view and interact with their own records.

Foreign key constraints with ON DELETE CASCADE ensure that deleting a user account automatically removes all associated data, preventing orphaned records. At the application level, authenticated user identity is stored in the session and used to filter all database retrieval operations. This dual-layer approach ensures isolation without requiring complex database-level permission schemes. PostgreSQL Row-Level Security (RLS) is identified as a future enhancement for additional defence in depth independent of application logic.

5. Visualisations

The platform provides a set of analytical visualisations, with core outputs mainly generated server-side using Matplotlib as static, publication-ready PNG files stored in src/analysis_MPL/plots. Supplementary visualisations are rendered interactively using Plotly, with all interactive figures offering a static snapshot download option (for more details about the libraries used refer back to section 3.3).

5.1 Top Performers Table

The Top-10 Performers table shows the ten highest-performing variants within a single experiment, based on unified activity metrics and total mutation count relative to the WT. The interactive table includes the generation number, plasmid variant index, and the ability to expand full variant details: variant ID, parent ID, activity score, DNA and protein yields, QC validation notes, and mutation details including type, amino acids, nucleotides, and position.

top10_variants

variant_id	generation_number	plasmid_variant_index	activity_score	total_mutations
42370	10	273	4.141854932539928	24.0
42385	10	288	3.7789854679512063	34.0
42392	10	295	3.6707249390573358	22.0
42390	10	293	3.617234095956874	20.0
42369	10	272	3.585426352979138	22.0
42381	10	284	3.5837891476565327	22.0
42368	10	271	3.5124631312657932	21.0
42379	10	282	3.477411249142448	27.0
42374	10	277	3.4385647261581713	22.0
42391	10	294	3.3921852551133025	22.0

Figure 3. Top 10 Performing variants table from example data.

5.2 Activity Score Distribution by Generation Plot

The activity score distribution is visualised as a violin plot across all generations of the experiment, allowing observation of variability, emergence of high-performing subpopulations, and performance improvement trends. Central tendency shifts, changes in performance spread, and appearance of outliers are all interpretable against the WT control baseline of 1.0. (red).

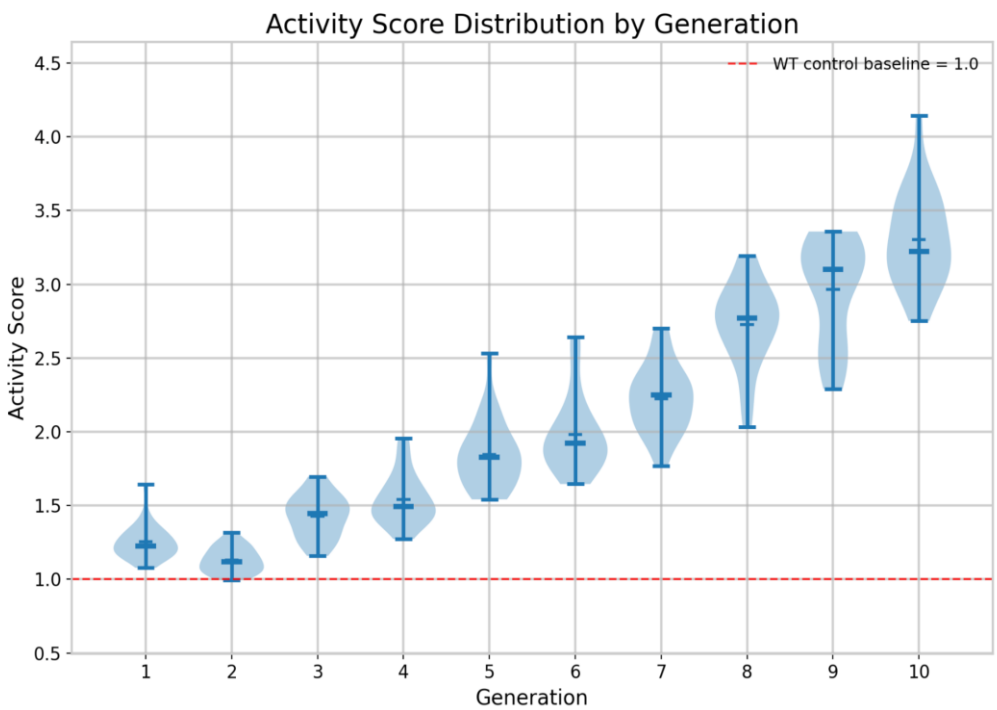

Figure 4. Activity score distribution by Generation from example data.

Activity score increases as generation increases, positive correlation.

5.3 Variant Lineage Visualisation

The variant lineage graph illustrates evolutionary relationships between variants across generations. Each variant is represented as a node; directed edges indicate parent-child relationships based on parent_variant_id. Node attributes such as generation number and activity score are encoded visually. An upward trend across generations indicates successful directed evolution; a downward pattern indicates unsuccessful variants. These results can be further identified with the p-value (beneficial mutations occurring or, suggesting random variation rather than meaningful evolution) and Pearson correlation coefficient (r), which could indicate a positive, negative or no correlation between generation and activity score.

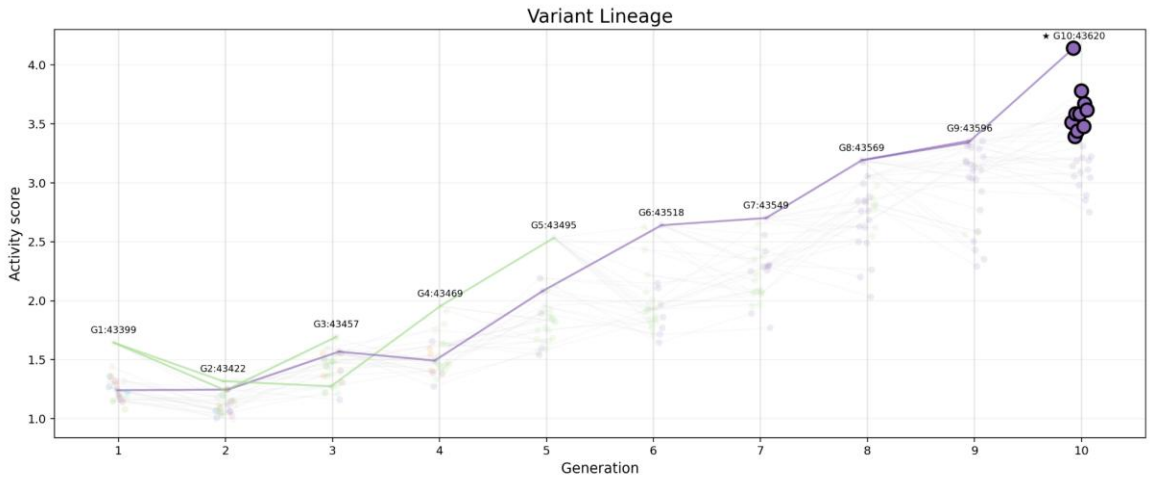

Figure 5. Variant lineage graph from example data.

Positive upward trend: as generation increases, the activity score also increases in generation 10 (especially for the first best performing variant). Performance in enzyme efficiency has evolved from parent to child up to the latest 10th generation.

5.4 Protein Similarity Network (Mutation Co-occurrence)

The protein similarity network provides a structural view of relationships between variants based on co-occurrence of protein mutations. Each node represents a variant; edges connect variants sharing common mutations. Connection strength is reflected by the Jaccard index (0 = no shared mutations, 1 = identical mutation sets). The network enables identification of clusters of variants following similar mutational trajectories, potentially indicating convergent evolution, shared functional mechanisms, or recurring beneficial mutation combinations.

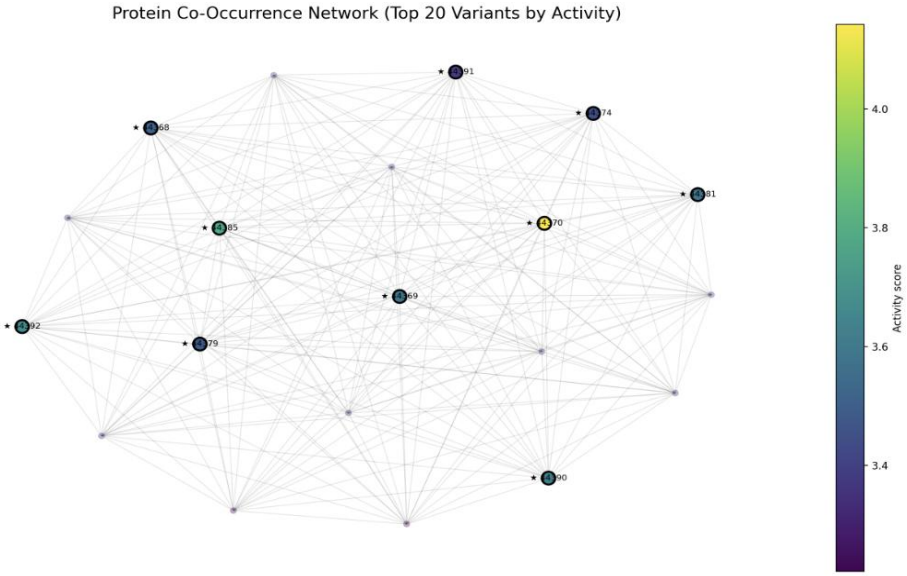

Figure 6. Protein co-occurrence network for the top 20 variants by activity score from example data.

The nodes indicate the variants, highlighted ones show the top 10 best performing variants, while edges show the connections by mutation co-occurrence.

5.5 Bonus Visualisations: Multi-Scale Analytical Narrative

The bonus visualisations are organised within a hierarchical multi-scale analytical framework that spans four interconnected levels: phenotype space, domain, residue, and variant. Together, this structured approach enables a comprehensive interpretation of the relationships between global sequence-activity patterns and the specific mutational architectures of individual clones.

5.5.1 Phenotype - Space level: 3D activity landscape

The 3D activity landscape illustrates the relationship between sequence diversity and functional performance across analysed variants. Sequence features are reduced to two dimensions using methods like Principal Component Analysis (PCA) or t-distributed Stochastic Neighbour Embedding (t-SNE) for the X and Y axes, while the Z axis shows each variant's Activity Score. Points represent sequenced variants, and the interpolated surface offers a continuous activity view across sequence space. Clusters of high-activity points highlight regions where similar variants perform well, whereas isolated peaks indicate rare, highly active variants. PCA preserves global sequence variation structure, while t-SNE focuses on local clustering but may distort overall distances.

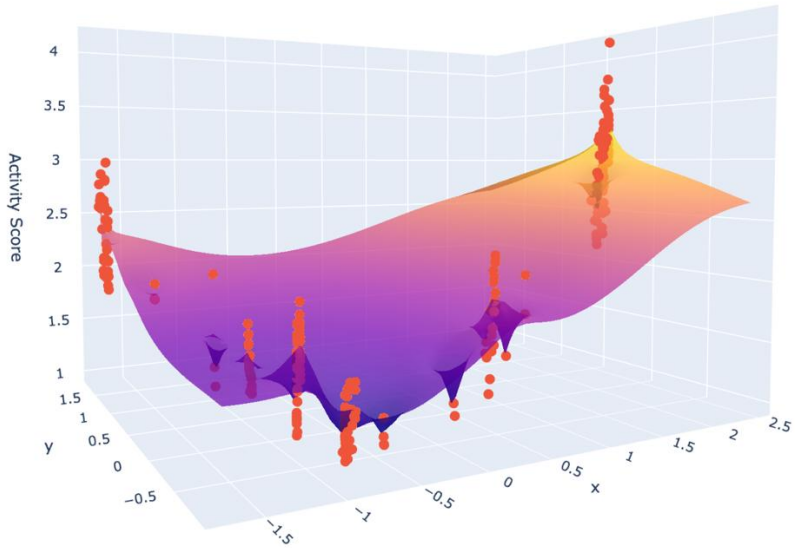

Figure 7 - The 3D activity landscape

5.5.2 Domain level: Domain enrichment heatmap

The domain-level mutation heatmap summarises non-synonymous mutation distribution across annotated protein domains over successive directed evolution generations. Rows represent UniProt-annotated domains, and columns correspond to experimental generations. Colour intensity and numerical values show the total mutations per domain per generation, enabling quick identification of frequently mutated domains. High mutation counts suggest adaptive modification, while consistently low counts indicate structural or functional constraints. Linking mutation accumulation to protein domains provides insight into sequence changes related to the polymerase's functional regions.

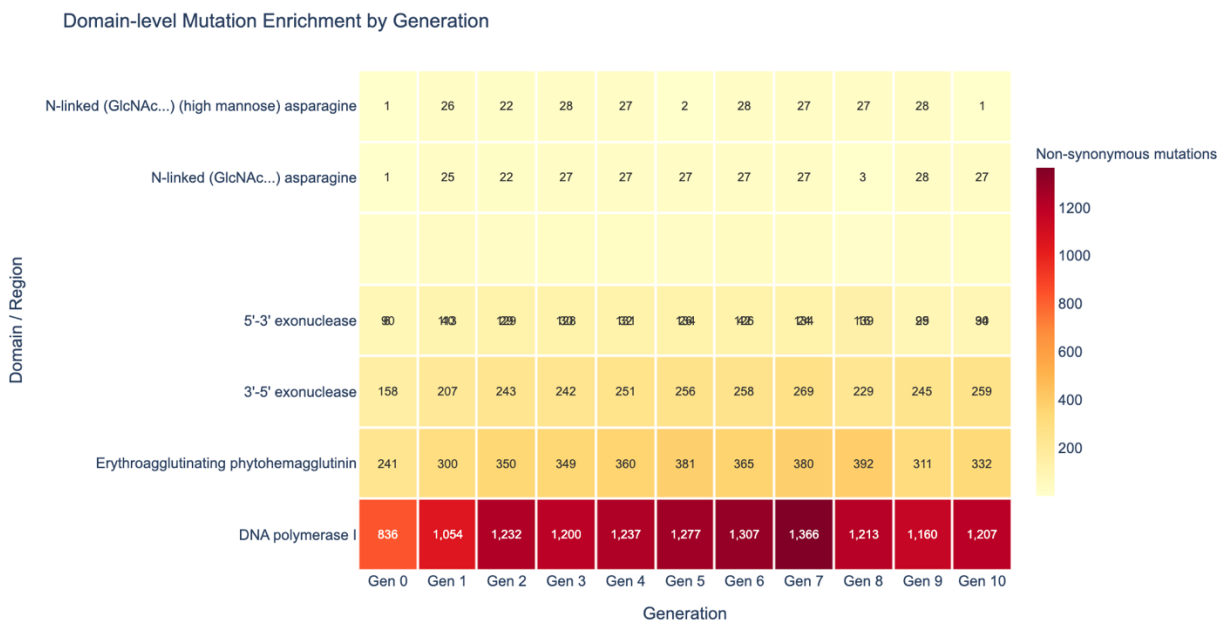

Figure 8 - Domain enrichment heatmap

5.5.3 Variant Level : Mutation fingerprinting

The mutation fingerprint visualisation maps amino acid substitutions for a selected variant onto the linear wild-type protein sequence. Each mutation is shown as a marker positioned according to its residue index, with the colour indicating the generation in which the mutation first appeared. This representation enables tracking of mutation accumulation across evolutionary rounds and highlights regions of the protein undergoing adaptive change. By visualising the spatial distribution of substitutions, the plot facilitates the identification of mutational hotspots and potential structure–function relationships.

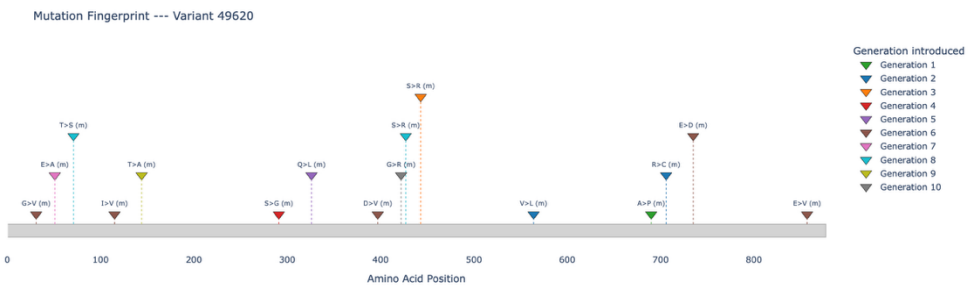

Figure 9 – Mutation fingerprinting

5.5.4 Residual Level: Mutation Frequency

Mutation frequency analysis quantifies the occurrence of non-synonymous mutations at each amino acid position across all analysed variants. Positions with high mutation counts indicate mutational hotspots, which may correspond to regions undergoing adaptive change during directed evolution, while positions with few or no mutations suggest functional constraints. The upper panel displays the total mutation frequency aggregated across all generations, whereas the lower panel breaks down mutation counts by generation, allowing examination of changes in mutational patterns over successive rounds of evolution. Interpretation should consider cohort composition and sequencing depth, as rare variants or uneven generation sizes may affect observed frequencies.

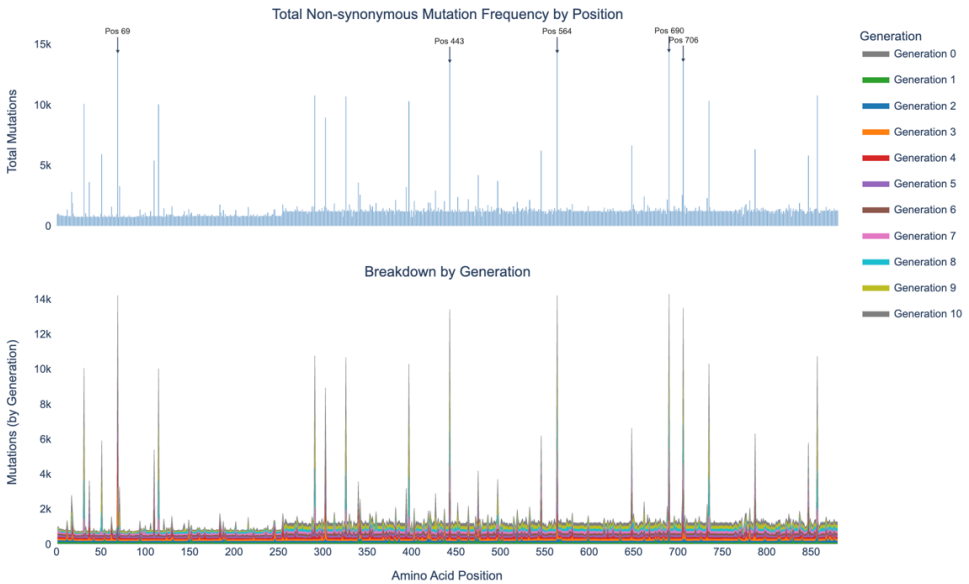

Figure 10. – Mutation Frequency

6. Limitations of the Software

6.1 Technical Limitations and Short-Term Improvements

Plasmid Validation Conservatism

The validator requires an exact WT match by default, rejecting plasmids with minor sequencing artefacts or annotation mismatches even when alignment identity and coverage are high. This reduces false positives at the cost of usability for imperfect constructs.

Planned improvement: Add a configurable tolerant validation mode with explicit identity and coverage thresholds, retaining strict mode as the default and recording thresholds and outcomes for auditability.

UniProt Annotation Staleness

UniProt feature annotations are inserted only when absent and are not refreshed on subsequent fetches; changes in UniProt curation do not automatically propagate to existing records.

Planned improvement: Implement a controlled refresh strategy using an explicit refresh action or an upsert-by-feature-key approach that updates only changed annotations while maintaining stability.

Login Timing Vulnerability

A known timing vulnerability exists in the login flow: skipping bcrypt when a username is not found produces a faster response that could theoretically allow timing-based username enumeration.

Planned improvement: Implement a dummy bcrypt check on failed username lookups to equalise response time regardless of whether the username exists.

Missing Maximum Password Length

Maximum password length is not currently enforced, leaving the application potentially vulnerable to bcrypt-based denial-of-service attacks via extremely long password inputs.

Planned improvement: Enforce a maximum password length of 64–128 characters.

Session Storage

Sessions use Flask's default signed-cookie storage, which does not support session invalidation across devices and limits scalability under high concurrency.

Planned improvement: Migrate to Redis-backed server-side sessions, enabling logout from all devices and improved scalability.

Materialised View Refresh

Materialised views must be manually refreshed after data loading or analysis steps; stale views may temporarily produce inconsistent visualisation outputs.

Planned improvement: Implement scheduled or triggered automatic refresh of materialised views following data loading or analysis completion.

No Retry Logic for Database Failures

Sequence processing writes results in batches within a single transaction. If PostgreSQL becomes temporarily unavailable during a flush, the entire batch is rolled back. The pipeline does not retry failed writes or resume from the last successful batch, reducing robustness under unstable network conditions.

Planned improvement: Implement retry logic with exponential backoff for batch persistence operations.

Single-Threaded Variant Execution

Variant processing is performed serially within a single background worker thread to avoid Python GIL overhead. Processing time therefore increases linearly with the number of variants, limiting throughput for very large libraries.

Planned improvement: Employ distributed task execution using Celery with Redis to enable parallel processing for large variant libraries.

Quality Control Summaries

Quality control summaries are currently available only in exported files and are not directly visible within the web interface during or after data upload.

Planned improvement: Expand quality control summaries to display directly in the web interface to provide immediate feedback during the ingestion workflow.

6.3 Scalability Considerations and Infrastructure Roadmap

Infrastructure

The current deployment uses a single PostgreSQL instance shared via Tailscale, which does not provide high availability, automated backups, or horizontal scaling for production use.

Long-term opportunity: Migrate to a managed cloud database service and containerise the application using Docker to simplify deployment, environment reproducibility, and operational monitoring.

Computational Throughput

Six-frame translation and BLOSUM62 alignment are computationally intensive for large variant libraries. Static Matplotlib figures are regenerated on each analysis trigger without caching, which could become a bottleneck for large experiments.

Long-term opportunity: Implement asynchronous task queuing (e.g., Celery with Redis) for computationally intensive analysis tasks, and add incremental or cached figure regeneration.

Visualisation Performance

The 3D activity landscape and detailed heatmaps can become slow or unresponsive for very large datasets, reducing interactivity and potentially requiring resolution reduction that obscures fine-grained patterns.

Long-term opportunity: Implement progressive loading and server-side data aggregation to maintain interactivity at scale without sacrificing analytical detail.

Advanced Analytics

Future analytical enhancements should include: fitness landscape modelling and regression to identify evolutionary trends and predict beneficial mutations; machine learning models to predict variant performance from sequence features; and mapping mutations onto protein structural models via integration with AlphaFold structure predictions. Biological integration opportunities include connection to laboratory information management systems (LIMS) and automated detection of convergent evolution patterns surfaced as analytical insights.

7. Conclusion

Overall, the Direct Evolution Monitoring Portal provides a complete, integrated solution, grounded in scientific correctness, modularity, user-centred simplicity and robust security, to support direct evolution analysis from raw experimental data.

By ensuring quality control layer and biological plausibility at every stage, the core analytical pipeline is built on establishing bioinformatics algorithms and standards with row-indexed, actionable error messages that support efficient data correction without discarding potentially valuable partial results.

8. References

Chang, J., Chapman, B., Friedberg, I., Hamelryck, T., Hoon, M. D., Cock, P., Antao, T., Talevich, E., & Wilczy´Nski, B. (2023). *BioPython tutorial and cookbook*. Available at : https://biopython.org/DIST/docs/tutorial/Tutorial-1.82.pdf (Accessed: 01 March 2026)

Currin, A., Parker, S., Robinson, C.J., Takano, E., Scrutton, N.S. and Breitling, R. (2021). The evolving art of creating genetic diversity: From directed evolution to synthetic biology. *Biotechnology Advances*, 50, p.107762.doi:https://doi.org/10.1016/j.biotechadv.2021.107762.

Gu, Z., Eils, R., Schlesner, M., & Ishaque, N. (2018). EnrichedHeatmap: an R/Bioconductor package for comprehensive visualization of genomic signal associations. *BMC Genomics*, *19*(1), 234. doi: https://doi.org/10.1186/s12864-018-4625-x

Mead, D.J., Simón Lunagomez and Gatherer, D. (2019). Visualization of protein sequence space with force-directed graphs, and their application to the choice of target-template pairs for homology modelling. *Journal of Molecular Graphics & Modelling*, [online] 92, p.180. doi:https://doi.org/10.1016/j.jmgm.2019.07.014.

Mirzaei, G. (2023). Constructing gene similarity networks using co-occurrence probabilities. *BMC Genomics*, 24(1). doi:https://doi.org/10.1186/s12864-023-09780-w.

Morais, P., Adachi, H. and Yu, Y.T.(2020). Suppression of nonsense mutations by new emerging technologies. *International Journal of Molecular Sciences*, *21*(12), p.4394. doi:https://doi.org/10.3390/ijms21124394

Oelschlaeger, P.(2024) Molecular mechanisms and the significance of synonymous mutations. *Biomolecules*, *14*(1), p.132. doi: https://doi.org/10.3390/biom14010132

Sievers, F. and Higgins, D.G.(2018) Clustal Omega for making accurate alignments of many protein sequences. *Protein Science*, *27*(1), pp.135-145.doi: https://doi.org/10.1002/pro.3290

Yu, R., Huang, Z., Lam, T.Y. and Sun, Y. (2024) Utilizing profile hidden Markov model databases for discovering viruses from metagenomic data: A comprehensive review. *Briefings in Bioinformatics*, *25*(4), p.bbae292. doi: https://doi.org/10.1093/bib/bbae292